

1. 2. 操作结构体

```
struct Op {  
    int type, a, b;  
};
```

这个结构体存原始操作。

操作有两种：

```
1 a b    加桥  
2 a      查询
```

所以：

```
type
```

表示操作类型。

```
a
```

如果是加桥，表示连接国家 `a` 和 `a + 1`。

如果是查询，表示从国家 `a` 出发。

```
b
```

只有加桥操作才有意义，表示桥的高度。

3. 当前块内新增桥结构体

```
struct Add {  
    int a, b, id;  
    int u, v;  
};
```

这个结构体专门存**当前块内的加桥操作**。

含义如下：

```
a, b
```

表示原始桥：

```
高度 b, 连接国家 a 和 a + 1  
id
```

表示这座桥是第几个操作加入的。

因为当前块内有些桥可能在某个查询之后才加入，所以查询时要判断：

```
if (e.id >= i) continue;
```

意思是：这座桥在当前查询之后才出现，不能算。

```
u, v
```

这是最关键的两个量。

它们不是 a 和 $a + 1$ 。

而是：

在只考虑旧桥的世界里，高度 b 时，位置 a 和 $a + 1$ 上分别是哪两条旧路径。

当前块内这座新桥的效果就是交换旧路径 u 和旧路径 v 。

4. 主函数与读入

```
int main() {
    cin.tie(nullptr)->sync_with_stdio(false);

    int n, m, q;
    cin >> n >> m >> q;
```

`cin.tie(nullptr)->sync_with_stdio(false);` 是快速输入输出。

读入：

```
n = 国家数量
m = 高度数量
q = 操作数量
```

```
vector<Op> op(q + 1);
for (int i = 1; i <= q; i++) {
    cin >> op[i].type >> op[i].a;
    if (op[i].type == 1) {
        cin >> op[i].b;
    }
}
```

这里把所有操作先读进来。

为什么要离线读入？

因为我们要按操作序列分块。

如果不提前知道一个块里有哪些操作，就没法预处理当前块内的新桥。

`op` 下标从 1 开始，方便和题目中的第 i 个操作对应。

5. 块大小

```
const int S = 450;
```

`S` 是分块大小。

也就是说，每 450 个操作作为一块。

这题总复杂度大概是：

```
O(qS + (n + m + q)q / S)
```

`q <= 1e5`，取 `S` 在 300 ~ 700 之间都可以。

这里用 450 是一个比较稳的经验值。

6. 旧桥、位置数组、答案排列

```
vector<vector<int>> old(m + 1);  
vector<int> pos(n + 1), f(n + 1);
```

这三个变量非常重要。

`old[h]`

```
vector<vector<int>> old(m + 1);
```

`old[h]` 存的是：

当前块之前，已经加入的、位于高度 `h` 的所有桥。

如果 `old[h]` 里面有一个数 `a`，表示旧桥：

```
高度 h，连接 a 和 a + 1
```

例如：

```
old[5] = {2, 7};
```

表示高度 5 有两座旧桥：

```
2 -- 3  
7 -- 8
```

pos[i]

```
vector<int> pos(n + 1);
```

pos[i] 表示:

在只考虑旧桥的模拟过程中, 当前高度下, 位置 i 上是哪条原始路径。

一开始没有走任何桥:

```
pos[1] = 1  
pos[2] = 2  
pos[3] = 3  
...
```

如果遇到一座旧桥 $a \rightarrow a + 1$, 就执行:

```
swap(pos[a], pos[a + 1]);
```

因为这座桥会让两个位置上的路径交换。

f[x]

```
vector<int> f(n + 1);
```

f[x] 表示:

只走旧桥时, 从国家 x 出发, 最终到达哪个国家。

模拟完全部高度后, 如果:

```
pos[i] = x
```

说明原来从国家 x 出发的路径最终到了位置 i。

所以:

```
f[x] = i;
```

代码里写成:

```
for (int i = 1; i <= n; i++) {  
    f[pos[i]] = i;  
}
```

7. 开始分块

```
for (int l = 1; l <= q; l += S) {  
    int r = min(q, l + S - 1);
```

这里枚举每一块。

当前块的操作编号范围是：

[l, r]

比如 `S = 450`：

第 1 块：1 ~ 450
第 2 块：451 ~ 900
第 3 块：901 ~ 1350
...

处理每一块时，有一个重要原则：

块之前的桥放进 `old`，当成固定背景；当前块里的桥放进 `add`，查询时暴力扫。

8. 收集当前块内新增桥

```
vector<Add> add;  
for (int i = l; i <= r; i++) {  
    if (op[i].type == 1) {  
        add.push_back({op[i].a, op[i].b, i, 0, 0});  
    }  
}
```

这里把当前块里的所有加桥操作收集起来。

比如当前块中有：

操作 12: 1 3 5
操作 20: 1 7 2
操作 31: 1 4 8

那么 `add` 里会有三座新桥：

(a=3, b=5, id=12)
(a=7, b=2, id=20)
(a=4, b=8, id=31)

此时 `u, v` 先设为 0，后面会在旧桥世界中算出来。

9. 按高度排序当前块内的新桥

```
sort(add.begin(), add.end(), [&](const Add &x, const Add &y) {
    if (x.b != y.b) {
        return x.b < y.b;
    }
    return x.id < y.id;
});
```

为什么要按高度排序？

因为英雄走路时，经过桥的顺序由高度决定，不由操作时间决定。

所以对于一次查询，当前块内已经存在的新桥要按照：

高度从小到大

处理。

如果两个桥高度相同，它们不会相邻，顺序无所谓。

这里用 `id` 作为第二关键字，只是让排序稳定、确定。

10. 初始化旧桥模拟

```
iota(pos.begin(), pos.end(), 0);
```

`iota` 会把数组填成：

```
pos[0] = 0
pos[1] = 1
pos[2] = 2
...
pos[n] = n
```

我们只用 `1 ~ n`。

含义是：

在高度 1 之前，每个国家上的路径就是自己。

```
int p = 0;
```

`p` 是指针，指向当前还没处理的 `add[p]`。

因为 `add` 已经按照高度排序了，所以我们可以一边扫高度 `h`，一边处理高度为 `h` 的新桥。

11. 重构旧桥世界，同时计算新桥对应的路径交换

这是整份代码最核心的一段：

```
for (int h = 1; h <= m; h++) {
    while (p < int(add.size()) && add[p].b == h) {
        int a = add[p].a;
        add[p].u = pos[a];
        add[p].v = pos[a + 1];
        p++;
    }

    for (auto a : old[h]) {
        swap(pos[a], pos[a + 1]);
    }
}
```

我们逐层高度 `h = 1 ~ m` 扫描。

11.1 先处理当前高度的新桥信息

```
while (p < int(add.size()) && add[p].b == h) {
    int a = add[p].a;
    add[p].u = pos[a];
    add[p].v = pos[a + 1];
    p++;
}
```

注意：这里不是把新桥真的加入模拟。

这里只是计算：

在旧桥世界里，这座新桥连接的两端分别是哪两条路径。

如果新桥是：

`1 a h`

也就是高度 `h`，连接位置 `a` 和 `a + 1`。

当前旧桥模拟中的：

`pos[a]`

表示位置 `a` 上是哪条路径。

`pos[a + 1]`

表示位置 `a + 1` 上是哪条路径。

所以这座新桥等价于：

```
swap(pos[a], pos[a + 1]);
```

但是因为它是当前块内的新桥，不能加入旧桥模拟。

我们只记录：

```
add[p].u = pos[a];  
add[p].v = pos[a + 1];
```

以后查询时，如果当前路径编号是 `u`，就变成 `v`；如果是 `v`，就变成 `u`。

11.2 再执行旧桥交换

```
for (auto a : old[h]) {  
    swap(pos[a], pos[a + 1]);  
}
```

`old[h]` 里存的是块之前已经存在的旧桥。

这些桥在整个当前块里都一定存在，所以它们要参与旧桥世界的模拟。

遇到高度 `h` 的旧桥 `a -- a + 1`，就交换：

```
pos[a], pos[a + 1]
```

这和梯子游戏完全一致。

11.3 为什么新桥记录在旧桥交换之前？

这一点容易疑惑。

在高度 `h`，如果既有旧桥，也有当前块的新桥，代码是：

```
先记录新桥的 u, v  
再执行旧桥交换
```

这是可以的。

原因是题目保证新加的桥不会和已有桥相连。

所以同一高度下，新桥和旧桥不可能共享端点。

例如旧桥在 `2 -- 3`，那么新桥不能在：

```
1 -- 2  
2 -- 3  
3 -- 4
```


它只能在不相邻的位置，比如 5 -- 6。

因此同一高度的交换彼此独立，先后顺序不影响结果。

12. 计算旧桥最终映射 `f`

```
for (int i = 1; i <= n; i++) {  
    f[pos[i]] = i;  
}
```

前面已经只用旧桥模拟完了全部高度。

此时：

```
pos[i]
```

表示最终位置 `i` 上是哪条原始路径。

如果：

```
pos[i] = x
```

说明从国家 `x` 出发，只经过旧桥，最后到了国家 `i`。

所以：

```
f[x] = i;
```

举个例子：

```
最终 pos:  
位置: 1 2 3 4  
路径: 3 1 4 2
```

含义是：

```
路径 3 最终到 1  
路径 1 最终到 2  
路径 4 最终到 3  
路径 2 最终到 4
```

所以：

```
f[3] = 1  
f[1] = 2  
f[4] = 3  
f[2] = 4
```

13. 处理当前块内的询问

```
for (int i = l; i <= r; i++) {
    if (op[i].type == 2) {
        int x = op[i].a;

        for (auto &e : add) {
            if (e.id >= i) {
                continue;
            }
            if (x == e.u) {
                x = e.v;
            } else if (x == e.v) {
                x = e.u;
            }
        }

        cout << f[x] << '\n';
    }
}
```

这是回答查询的部分。

13.1 查询起点

```
int x = op[i].a;
```

查询 `2 a`，表示从国家 `a` 出发。

这里令：

```
x = a
```

但注意，此时 `x` 的含义不是“当前位置”，而是：

当前正在沿着旧桥世界中的哪条路径走。

一开始当然是路径 `a`。

13.2 扫描当前块内的新桥

```
for (auto &e : add) {
```

`add` 已经按高度从小到大排序。

所以这里扫描顺序就是英雄真正遇到新桥的高度顺序。

13.3 跳过查询之后才加入的桥

```
if (e.id >= i) {
    continue;
}
```

`e.id` 是这座桥的操作编号。

当前查询是第 `i` 个操作。

如果：

```
e.id >= i
```

说明这座桥在当前查询时还没加入，不能算。

只处理：

```
e.id < i
```

的桥。

13.4 新桥对路径编号的影响

```
if (x == e.u) {
    x = e.v;
} else if (x == e.v) {
    x = e.u;
}
```

当前块内的一座新桥等价于交换旧路径：

```
u <-> v
```

如果当前走的是路径 `u`，遇到这座桥后切换到路径 `v`。

如果当前走的是路径 `v`，遇到这座桥后切换到路径 `u`。

如果当前路径不是 `u` 也不是 `v`，说明这座桥不在当前路径上，对它没影响。

13.5 输出最终答案

```
cout << f[x] << '\n';
```

扫描完当前块内所有已经存在的新桥后，`x` 表示：

经过这些新桥切换之后，最终会沿着旧桥世界中的路径 `x` 继续走。

而旧桥世界中，路径 `x` 最后到达：

```
f[x]
```

所以答案就是：

```
f[x]
```

14. 当前块处理完后，把新桥变成旧桥

```
for (int i = 1; i <= r; i++) {  
    if (op[i].type == 1) {  
        old[op[i].b].push_back(op[i].a);  
    }  
}
```

这一块处理完之后，当前块内所有加过的桥，在下一块看来都已经是旧桥了。

所以把它们加入 `old`。

例如操作：

```
1 a b
```

表示高度 `b` 加桥 `a -- a + 1`，所以：

```
old[b].push_back(a);
```

下一块重构时，它就会参与旧桥世界模拟。

15. 代码整体流程总结

整份代码可以概括为：

读入所有操作

`old = 空`

按块处理操作：

收集当前块内所有加桥操作 `add`

按高度排序 `add`

只用 `old` 重构旧桥世界：

`pos[i] = i`

从高度 `1` 到 `m` 扫描：

对当前高度的新桥，记录它交换哪两条旧路径 `u, v`

对当前高度的旧桥，执行 `swap(pos[u], pos[v])`

得到 `f[x] = 旧桥世界中路径 x 的终点`

回答当前块内查询：

x = 查询起点

扫描当前块内已经出现的新桥：

如果 x 是 u 或 v ，就切换

输出 $f[x]$

当前块结束：

把当前块内新增桥加入 old

16. 一个小例子帮助理解 u, v

假设只考虑旧桥时，某个高度之前：

```
位置：    1  2  3  4
路径编号：3  1  4  2
```

现在当前块内有一座新桥：

```
1 2 h
```

也就是高度 h ，连接位置 2 和 3 。

那么：

```
u = pos[2] = 1
v = pos[3] = 4
```

所以这座桥不是交换国家 2 和 3 ，而是交换旧路径：

```
1 <-> 4
```

查询时如果当前路径编号是 1 ，经过这座桥就变成 4 。

如果当前路径编号是 4 ，就变成 1 。

17. 最容易写错的地方

第一，查询时扫描的是 add ，而且必须按高度顺序扫描，不能按操作顺序扫描。

第二，新桥的 u, v 要在旧桥世界里算出来，不是直接 $u = a, v = a + 1$ 。

第三，当前块的新桥不能加入 pos 的模拟，否则会被重复计算。

第四，处理查询时必须判断：

```
if (e.id >= i) continue;
```

否则会把未来才加入的桥提前算进去。

18. 完整代码再看一遍

```
#include <bits/stdc++.h>
using namespace std;

using i64 = long long;

struct Op {
    int type, a, b;
};

struct Add {
    int a, b, id;
    int u, v;
};

int main() {
    cin.tie(nullptr)->sync_with_stdio(false);

    int n, m, q;
    cin >> n >> m >> q;

    vector<Op> op(q + 1);
    for (int i = 1; i <= q; i++) {
        cin >> op[i].type >> op[i].a;
        if (op[i].type == 1) {
            cin >> op[i].b;
        }
    }

    const int S = 450;

    vector<vector<int>> old(m + 1);
    vector<int> pos(n + 1), f(n + 1);

    for (int l = 1; l <= q; l += S) {
        int r = min(q, l + S - 1);

        vector<Add> add;
        for (int i = l; i <= r; i++) {
            if (op[i].type == 1) {
                add.push_back({op[i].a, op[i].b, i, 0, 0});
            }
        }

        sort(add.begin(), add.end(), [&](const Add &x, const Add &y) {
            if (x.b != y.b) {
                return x.b < y.b;
            }
            return x.id < y.id;
        });
    }
```

```

iota(pos.begin(), pos.end(), 0);

int p = 0;
for (int h = 1; h <= m; h++) {
    while (p < int(add.size()) && add[p].b == h) {
        int a = add[p].a;
        add[p].u = pos[a];
        add[p].v = pos[a + 1];
        p++;
    }

    for (auto a : old[h]) {
        swap(pos[a], pos[a + 1]);
    }
}

for (int i = 1; i <= n; i++) {
    f[pos[i]] = i;
}

for (int i = 1; i <= r; i++) {
    if (op[i].type == 2) {
        int x = op[i].a;

        for (auto &e : add) {
            if (e.id >= i) {
                continue;
            }
            if (x == e.u) {
                x = e.v;
            } else if (x == e.v) {
                x = e.u;
            }
        }

        cout << f[x] << '\n';
    }
}

for (int i = 1; i <= r; i++) {
    if (op[i].type == 1) {
        old[op[i].b].push_back(op[i].a);
    }
}

return 0;
}

```

一句话记住这份代码：

`old` 负责压缩块前所有桥，`add` 负责暴力处理块内新桥，`pos` 用来模拟旧桥路径，`f` 用来把最终路径编号转成答案。

